

Оптимизация многопоточных программ

Матвей Ефимов

Эксперт в Go, Python, SQL



Матвей Ефимов

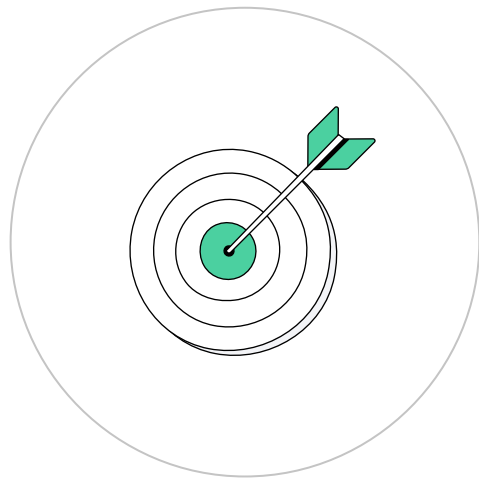
О спикере:

- Преподаватель с более чем 5-летним опытом в IT-образовании
- Разработчик учебных программ и эксперт в Go, Python, SQL
- Фриланс-программист и наставник студентов от школ до вузов



Цели занятия

- 1 Как находить и анализировать узкие места в многопоточном коде
- 2 Какие ошибки чаще всего снижают производительность
- 3 Как диагностировать contention, лишние горутины и избыточную синхронизацию
- 4 Как выглядит эффективная оптимизация: кейс до/после
- 5 Какие практики стоит применять всегда

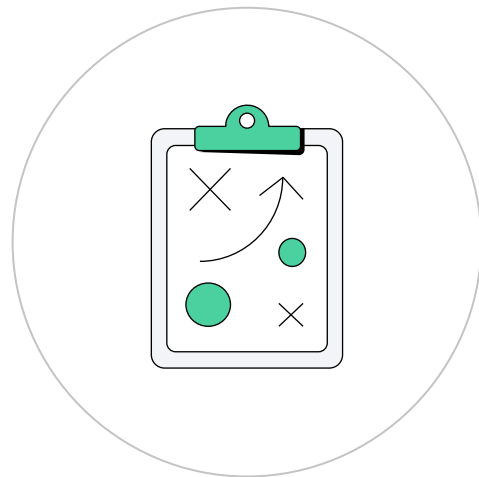


Вопрос к вам

- Кто сталкивался с проблемами производительности в многопоточном коде?
- Какая из целей занятия для вас самая актуальная?

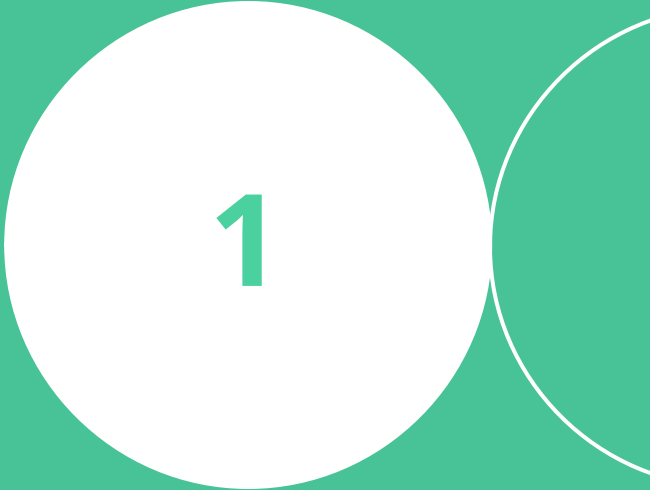
План занятия

- 1 Зачем нужна оптимизация многопоточных программ
- 2 Избыточная синхронизация
- 3 Оптимизация и лучшие практики
- 4 Конкуренция за ресурсы
- 5 Способы устранения и лучшие практики
- 6 Слишком много горутин
- 7 Подходы к решению и лучшие практики
- 8 Общие рекомендации и чеклист



*Нажмите на нужный раздел для перехода

Зачем нужна оптимизация многопоточных программ



1

Важность оптимизации многопоточности

Нужна, чтобы:

- задачи выполнялись быстрее за счёт эффективного параллелизма
- система сохраняла производительность при росте числа пользователей и объёма данных
- параллельный доступ к общим данным не приводил к ошибкам и потере данных

Скрытые проблемы многопоточных сервисов



Пример:

- Сервис на Go с сотнями горутин работает без ошибок в условиях разработки. При нагрузочном тестировании увеличение числа клиентов или данных приводит к торможению. Отсутствие ожидаемого ускорения указывает на скрытые проблемы в реализации многопоточности



Какие могут быть причины падения производительности?

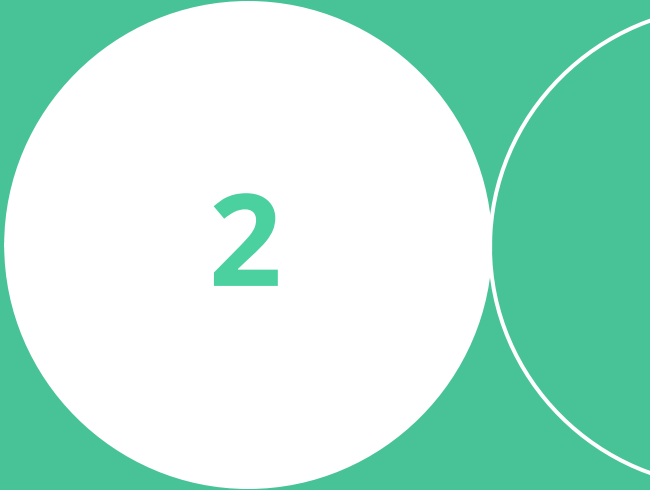
Возможные причины

- Неэффективная работа планировщика горутин
- Накладные расходы на синхронизацию
- Неравномерная загрузка ядер
- Конкуренция за общие ресурсы (память, дескрипторы)
- Грубая синхронизация в коде



Ваши вопросы?

Избыточная синхронизация



2

Что такое избыточная синхронизация

- Избыточная синхронизация — защита ресурсов в тех случаях, когда в ней нет необходимости или она охватывает слишком широкую область кода
- Используются мьютексы, каналы, атомики, condition переменные и другие средства, которые могли бы быть заменены более узкой локальной блокировкой или вовсе исключены
- Широкие блокировки увеличивают конкуренцию между горутинами и создают узкие места в производительности

Причины избыточной синхронизации

- Переоценка риска доступа к данным
- Стремление избежать **data race** любой ценой
- Использование мьютексов без необходимости
- Страх перед атомарностью — избегание **sync/atomic** даже в подходящих случаях.
Предпочтение тяжёлой синхронизации вместо атомарного счётчика или флага, что приводит к потере производительности

Почему синхронизация не бесплатна

- 1 Горутина захватывает мьютекс, чтобы выполнить операцию
- 2 Если другая горутина хочет сделать то же самое — она блокируется и переходит в ожидание
- 3 Планировщик сохраняет её состояние и переключает выполнение на другие задачи
- 4 Когда мьютекс освобождается, горутина возвращается в работу — это снова требует переключения контекста
- 5 Каждое переключение и ожидание добавляют накладные расходы и снижают общую производительность



**Какие виды блокировок вы используете
чаще всего в своём коде?**

Последствия грубой синхронизации

- 1 Грубая синхронизация мешает параллельному выполнению даже при корректной логике кода
- 2 Ресурсы блокируются чаще и дольше, чем это необходимо, и горютины начинают мешать друг другу
- 3 Производительность падает, масштабируемость снижается, ресурсы расходуются неэффективно

Типовые ошибки при работе с мьютексами

- **Проблема:** мьютекс блокирует каждое обращение, даже если это только чтение данных
- **Почему это плохо:**
 - Параллельные чтения не конфликтуют, но из-за мьютекса выполняются последовательно
 - Производительность падает при увеличении числа запросов
- **Чаще всего встречается:** работа со срезами и картами без разделения на чтение и запись

Как проявляется избыточная синхронизация

- Каждый доступ к счётчику блокируется мьютексом, даже если это простое чтение или прибавление единицы
- При большом числе горутин они выстраиваются в очередь, ожидая освобождения мьютекса
- Пока горутина ждёт своей очереди, процессор простаивает вместо выполнения полезной работы

Пример кода

Как можно переписать этот фрагмент, чтобы убрать лишние блокировки?

```
var mu sync.Mutex
var counter int

func increment() {
    mu.Lock()
    counter++
    mu.Unlock()
}
```

- Нет ошибки, но есть избыточная блокировка
- Частые вызовы приводят к снижению производительности

Лучшие альтернативы

- Использование атомарной операции для простых числовых данных
- Канал с обработчиком событий
- Локальная синхронизация отдельных полей вместо блокировки всей структуры
- Применение шардинга (разделение данных на независимые части)

Последствия избыточной синхронизации

- **Снижение параллелизма** — горутины ждут друг друга, вместо того чтобы выполняться одновременно
- **Рост задержек** — каждая операция тратит дополнительное время на ожидание блокировки
- **Перегрузка планировщика Go** — частые блокировки и пробуждения горутин отнимают ресурсы у полезных задач
- **Простой CPU** — процессор не загружен на 100%, даже если задачи есть
- **Нестабильная работа под нагрузкой** — система быстро «захлебывается» при увеличении числа пользователей или объёма данных

Пример избыточной синхронизации

- Чтение и запись блокируются одним мьютексом
- Частые вызовы Get замедляются без необходимости
- Лучше применить sync.RWMutex или атомарную замену карты при обновлении, если это возможно

```
type Store struct {  
    mu    sync.Mutex  
    items map[string]string  
}  
  
func (s *Store) Get(key string) string {  
    s.mu.Lock()  
    defer s.mu.Unlock()  
    return s.items[key]  
}  
  
func (s *Store) Set(key string, value string) {  
    s.mu.Lock()  
    defer s.mu.Unlock()  
    s.items[key] = value  
}
```

Почему возникает избыточная синхронизация

- Страх ошибок гонки данных (`data race`)
- Отсутствие глубокого понимания принципов мьютексов и работы планировщика
- Привычка синхронизировать любой подозрительный участок кода
- Нет регулярного профилирования и анализа производительности — не видны узкие места



Ваши вопросы?

Оптимизация и лучшие практики



3

Почему оптимизируем синхронизацию

- 1 Оптимизация не означает отказ от всех мьютексов — они остаются там, где это действительно нужно
- 2 Задача — заменить тяжёлые и общие блокировки на более точные и лёгкие механизмы: шардирование, каналы, локальные данные, RCU
- 3 Правильно подобранный механизм синхронизации снижает конкуренцию за ресурсы и повышает масштабируемость
- 4 Главное — сохранить корректность работы программы, убрав лишние задержки и узкие места

Основные подходы к устранению избыточной синхронизации

- Сужение области захвата мьютекса
- Замена мьютексов на RWMutex
- Использование атомарных операций
- Применение sharding — разделение данных на независимые блоки
- Использование каналов как альтернативы синхронизации
- Read-copy-update (RCU) и lock-free подходы при редких записях

Примеры

До и после оптимизации

Пример 1

Пример сужения области захвата мьютекса

- Исходный код блокирует мьютекс на всю функцию
- `time.Sleep` и вывод — вне доступа к разделяемым данным
- Мьютекс удерживается во время долгой обработки
- Горотины ждут, пока одна «спит»

Исходный код с избыточной синхронизацией

```
var mu sync.Mutex
var data []int

func process() {
    mu.Lock()
    defer mu.Unlock()

    // Чтение данных
    copyData := make([]int, len(data))
    copy(copyData, data)

    // Долгая обработка
    time.Sleep(100 * time.Millisecond)

    fmt.Println("Processed:", copyData)
}
```

Оптимизированный код с узкой областью захвата мьютекса

```
func process() {  
    mu.Lock()  
    copyData := make([]int, len(data))  
    copy(copyData, data)  
    mu.Unlock()  
  
    // Обработка — вне критической секции  
    time.Sleep(100 * time.Millisecond)  
  
    fmt.Println("Processed:", copyData)  
}
```

Преимущества оптимизации

- 1 Мьютекс захвачен только для чтения данных
- 2 Долгая обработка происходит без блокировки
- 3 Повышение параллелизма и производительности при высокой конкуренции

Пример 2

Проблема с mutex при частом чтении и редких записях

- Обычный sync.Mutex блокирует все операции, включая чтение
- При параллельных вызовах Get горютины выстраиваются в очередь
- Это снижает производительность и увеличивает задержки

Исходный код с sync.Mutex

```
type Store struct {  
    mu    sync.Mutex  
    data map[string]string  
}  
  
func (s *Store) Get(key string) string {  
    s.mu.Lock()  
    defer s.mu.Unlock()  
    return s.data[key]  
}  
  
func (s *Store) Set(key, val string) {  
    s.mu.Lock()  
    defer s.mu.Unlock()  
    s.data[key] = val  
}
```

Исходный код с sync.Mutex

Проблема чтения с обычным Mutex

- Метод Get() использует обычный Mutex
- Все горютины, вызывающие Get(), выстраиваются в очередь
- Чтение выполняется последовательно, а не параллельно

Неэффективность при частом чтении

- В реальных системах чтение значительно чаще записи
- Использование обычного Mutex заставляет горютины ждать
- Параллельное чтение невозможно, что снижает производительность

Пример 3

Проблема с использованием мьютекса для счётчика

- Частое обновление счётчика вызывает блокировки
- Полная блокировка ради простой операции дорогостоящая
- Производительность снижается при большом числе вызовов

Глобальная карта с одним мьютексом

Проблема № 1 — избыточная блокировка

- Для операции counter++ используется полноценный Mutex
- При большой нагрузке — тысячи горутин выстраиваются в очередь для инкрементации счётчика
- Возможен параллельный инкремент без блокировок, но используется последовательный доступ

```
var mu sync.Mutex
var counter int64

func Inc() {
    mu.Lock()
    counter++
    mu.Unlock()
}
```

Глобальная карта с одним мьютексом

Проблема № 2 — накладные расходы

- Вызовы Lock() и Unlock() требуют работы с планировщиком
- Могут возникать contention — конкуренция за ресурс между потоками
- Для простых атомарных операций существуют более эффективные альтернативы, чем Mutex

```
var mu sync.Mutex
var counter int64

func Inc() {
    mu.Lock()
    counter++
    mu.Unlock()
}
```

Оптимизация счётчика через atomic

Суть метода:

- Значение счётчика увеличивается за одно действие, без ожидания и блокировок
- Такой подход работает очень быстро, даже если обновлений много
- Особенно полезно для оперативного подсчёта числа запросов, событий, хитов и другой статистики

```
var counter int64

func Inc() {
    atomic.AddInt64(&counter, 1)
}
```

Пример 4. Sharding

- Проблема: глобальная карта с одним мьютексом блокирует все горютины
- Решение: разделить данные на несколько сегментов (shard'ов) с отдельными мьютексами
- Результат: параллельный доступ к разным shard'ам без взаимной блокировки, рост производительности при высокой нагрузке

Оптимизация через шардирование

```
type Shard struct {  
    mu    sync.RWMutex  
    data map[string]string  
}  
  
type Cache struct {  
    shards [16]Shard  
}  
  
func (c *Cache) getShard(key string)  
*Shard {  
    idx := hash(key) % len(c.shards)  
    return &c.shards[idx]  
}
```

```
func (c *Cache) Get(key string) string {  
    shard := c.getShard(key)  
    shard.mu.RLock()  
    defer shard.mu.RUnlock()  
    return shard.data[key]  
}  
  
func (c *Cache) Set(key, val string) {  
    shard := c.getShard(key)  
    shard.mu.Lock()  
    defer shard.mu.Unlock()  
    shard.data[key] = val  
}
```

Пример 5. Канал вместо мьютекса

- **Проблема:** очередь задач реализована через мьютекс
- **Недостатки:** лишняя синхронизация, сложность кода, низкая эффективность при большом числе горутин
- **Решение:** использовать канал как встроенный механизм синхронизации

Наивная реализация с мьютексом

```
var mu sync.Mutex
var tasks []string

func Add(task string) {
    mu.Lock()
    tasks = append(tasks, task)
    mu.Unlock()
}

func Get() string {
    mu.Lock()
    defer mu.Unlock()
    if len(tasks) == 0 {
        return ""
    }
    t := tasks[0]
    tasks = tasks[1:]
    return t
}
```

Наивная реализация с мьютексом

Проблема № 1 — глобальная блокировка

- Все операции Add и Get используют один мьютекс
- Даже при работе с разными элементами горютины выстраиваются в очередь
- При 100 параллельных вызовах Get задачи могли бы обрабатываться одновременно, но блокируются

Проблема № 2 — нет контроля за переполнением

- Частые вызовы Add приводят к бесконечному росту слайса tasks
- Риск утечки памяти
- Нет механизма ожидания при пустой или переполненной очереди

Проблема № 3 — сложность обработки пустой очереди

- Get возвращает пустую строку, если очередь пуста
- Такое поведение неочевидно
- Трудно отличить «пустую задачу» от ситуации, когда очередь пуста

Преимущества канала

- Синхронизация встроена в механизм channel
- Блокировка происходит только при пустой или полной очереди
- Код проще в поддержке и легче читается

```
var taskQueue = make(chan string, 100)

func Add(task string) {
    taskQueue <- task
}

func Get() string {
    return <-taskQueue
}
```

Пример 6. RCU — «ленивая» запись

Read-Copy-Update (RCU)

- Подходит, если чтений много, а записи редки
- Все читатели используют одну структуру без блокировок
- Запись создаёт новую копию и подменяет ссылку

Исходная реализация с мьютексом

Проблема № 1 — читатели блокируют друг друга

- Даже при использовании RWMutex горутини конкурируют за блокировку чтения
- Неэффективно при частых вызовах Get
- Особенно заметно, если данные почти не меняются

Проблема № 2 — запись блокирует все чтения

- При вызове Update берётся эксклюзивная блокировка
- Все операции Get ждут завершения записи
- Чтение могло бы продолжаться с предыдущей версией данных

```
type Config struct {  
    data map[string]string  
}
```

```
var mu sync.RWMutex  
var current *Config
```

Оптимизация через atomic.Pointer

```
var atomicConfig atomic.Pointer[Config]

func Get(key string) string {
    cfg := atomicConfig.Load()
    return cfg.data[key]
}

func Update(newData map[string]string) {
    newCfg := &Config{data: newData}
    atomicConfig.Store(newCfg)
}
```


Что изменилось и когда это эффективно

Чтение без блокировок (`atomic.Load`)

- `Get()` не использует мьютекс, а просто читает текущий указатель
- 1 000 горутин могут читать данные параллельно без конкуренции

Запись через атомарную подмену (`atomic.Store`)

- `Update()` создаёт новый `Config` и атомарно обновляет указатель
- Читатели работают со старой версией данных до следующего `Load()`

Эффективно, если:

- Данные читаются часто, а записываются редко (конфиги, кэши, `lookup`-таблицы)
- Структура большая и дорого копировать
- Нужна максимальная скорость чтения

Плюсы, ограничения и вывод

Плюсы:

- Чтение масштабируется линейно по числу ядер
- Нет блокировок при чтении — только атомарная загрузка указателя
- Простой код без `Lock()` / `Unlock()`

Ограничения:

- Не подходит для частых записей — нагрузка на GC
- Нет контроля за жизненным циклом старых данных — они остаются в памяти до очистки GC

Вывод:

- Используйте RCU при частых чтениях, редких записях и больших неизменяемых структурах
- Не используйте при частых изменениях данных или строгих требованиях к памяти



Ваши вопросы?

Конкуренция за ресурсы



4

Что такое contention

- Ситуация, когда несколько горутин или потоков одновременно пытаются получить доступ к одному ресурсу
- Результат — они конфликтуют, ожидая своей очереди
- Пример: одна кофемашина и 20 человек, все ждут, пока каждый сварит кофе
- В многопоточности это называют ресурсным конфликтом

Чем contention отличается от избыточной синхронизации

- Избыточная синхронизация — мы сами ограничиваем параллелизм больше, чем нужно
- Contention — естественное следствие ограниченности ресурса
- Даже при минимальной синхронизации множество горутин, обращающихся к одному объекту, создают «бутылочное горлышко»

Виды ресурсов, за которые возникает contention

- Мьютексы (`sync.Mutex`)
- `RWMutex` и другие блокировки — много читателей, но конкуренция за чтение
- Каналы (`chan`) — при большом числе одновременных записей или чтений
- Глобальные структуры данных — карта, очередь, пул соединений, логгер
- Внешние ресурсы — БД, файловая система, сеть
- CPU-ядра — особенно при привязке горутин к одному процессору Go или общему L1/L2 кэшу

Как проявляется contention

- Неожиданные тормоза при высокой параллельной нагрузке
- Программа «замирает» на пике числа клиентов
- В latency-тестах появляется «хвост» — часть запросов обрабатывается с задержкой
- CPU не загружен на 100%, хотя ресурсы есть
- В профилировании видно много ожиданий на мьютексах или блокировках
- Чем больше горутин — тем хуже ситуация, масштабируемость падает

Пример 1. Очередь задач

```
var mu sync.Mutex
var queue []string

func enqueue(task string) {
    mu.Lock()
    queue = append(queue, task)
    mu.Unlock()
}

func dequeue() string {
    mu.Lock()
    defer mu.Unlock()
    if len(queue) == 0 {
        return ""
    }
    task := queue[0]
    queue = queue[1:]
    return task
}
```

Пример 1. Очередь задач

- При 2–3 воркерах работает нормально
- При 100 воркерах — конкуренция за мьютекс, даже при пустой очереди
- Классический mutex contention: нагрузка выросла — производительность упала

Пример 2. Логгер

```
type Logger struct {  
    mu sync.Mutex  
    out io.Writer  
}  
  
func (l *Logger) Log(msg string) {  
    l.mu.Lock()  
    defer l.mu.Unlock()  
    l.out.Write([]byte(msg + "\n"))  
}
```

- При большом числе горутин (например, 500) все ждут мьютекс для записи
- Время отклика растёт
- Логгер становится узким местом, замедляя бизнес-логику

Пример 3. Глобальный счётчик

```
var mu sync.Mutex
var counter int

func increment() {
    mu.Lock()
    counter++
    mu.Unlock()
}
```

- На одной-двух горутинах всё быстро
- При сотнях клиентов каждая горутина ждёт мьютекс
- Время ожидания доступа измеряется миллисекундами, хотя операция — мгновенная

Почему contention коварен

На малых нагрузках и в тестах почти незаметен

Код может:

- работать быстро в dev-режиме
 - проходить все unit-тесты
 - отлично выглядеть при профилировании одиночного запроса
-
- На реальном трафике проявляется как медленная, нестабильная работа
 - Не ломает систему явно, но снижает её производительность изнутри

Связь contention и масштабируемости

- При наличии contention система не масштабируется горизонтально
- Увеличение числа воркеров, ядер, клиентов не даёт прироста производительности

Симптомы:

- рост задержек
- рост потребления CPU без пользы
- пики нагрузки и провалы в скорости работы

Даже при 64 ядрах и 2 000 горутинах может реально использоваться 2 ядра

Как отличить contention от других проблем

Высокая активность блокировок в pprof или traces

- Используйте go tool pprof с профилем блокировок (mutex, block)
- Видно, какие функции чаще ждут мьютекс
- Резкие изменения производительности при небольшом росте нагрузки
- Много горутин в состоянии waiting или runnable
- Одинаковые трассы ожидания — десятки горутин стоят в одном месте

Контеншен и блокировки — не одно и то же

Контеншен — шире, чем мьютексы

Может возникать при доступе к любому ресурсу:

- запись сотни горутин в один канал
- одновременная запись в одну строку БД
- использование общей структуры данных

Мьютексы — частный случай контеншена

Что дальше

- Уменьшить степень контеншена
- Перераспределить нагрузку по ресурсам
- **Использовать структурные приёмы:**
 - шардирование
 - каналы
 - копирование данных
- ✓ Переписать архитектуру для избегания конфликтов
- ✓ Разбор на примерах «до и после» с замерами производительности

Промежуточные итоги

- ✓ Contention — конфликт за ограниченные ресурсы
- ✓ Возникает даже при корректной многопоточности
- ✓ Главный симптом — падение производительности при росте параллельной нагрузки
- ✓ Причины не только в мьютексах, но и в структурах данных, каналах, файлах, логгерах, ядрах CPU
- ✓ Устранять contention нужно, но сначала важно его найти



Ваши вопросы?

Перерыв

10-15 минут



Способы устранения и лучшие практики



5

Зачем устранять contention

- Это про устойчивость, масштабируемость и эффективность
- Система, работающая при 10 клиентах, но «падающая» при 100, — пример неэффективного распределения ресурсов
- Нужно пересматривать архитектуру и синхронизацию
- Решение — переход от общего ресурса к распределённой нагрузке, от монолита к независимым рабочим единицам

Подходы к устранению contention

- Sharding — деление данных на независимые блоки
- Разделение каналов (channel per consumer / producer)
- Локализация данных — привязка к потоку или воркеру
- Буферизация
- Асинхронная обработка и очереди
- Read-Copy-Update (RCU) и lock-free подходы

Разделение каналов (channel per worker)

```
tasks := make(chan Task)

go func() {
    for {
        t := <-tasks
        process(t)
    }
}()
```

Недостатки одного канала

- Горячая точка (hot spot) — все горутины пишут в один канал, а читает одна → contention
- Нет параллелизма — задачи обрабатываются последовательно, даже если process() CPU-bound
- Буферизация не решает проблему — при высокой нагрузке канал всё равно блокирует отправителей

Как работает channel per worker

Каждому воркеру — свой канал

- 8 воркеров = 8 независимых каналов
- Нет общего канала → нет contention

Распределение через хеш

- $\text{hash}(\text{task.ID}) \% 8$ гарантирует, что одинаковые задачи попадут к одному воркеру (если нужно)
- Можно использовать `sync/atomic` для round-robin

Буферизация на уровне воркера

- Каждый канал имеет буфер (например, 100 задач) для сглаживания пиков

```
const workerCount = 8

var taskChans [workerCount]chan Task

func init() {
    for i := 0; i < workerCount; i++ {
        taskChans[i] = make(chan Task, 100)
        go worker(taskChans[i])
    }
}

func distribute(task Task) {
    idx := hash(task.ID) % workerCount
    taskChans[idx] <- task
}
```

Как работает channel per worker

Плюсы:

- Высокий параллелизм — задачи обрабатываются в workerCount горutiнах
- Нет блокировок при отправке — если воркеры успевают, отправители не ждут
- Локальность данных — можно привязать к воркеру (например, через sync.Pool)

Когда использовать:

- Очереди задач с высокой частотой добавления (например, HTTP-запросы)
- CPU-bound операции, которые можно распараллелить (кодирование/декодирование)
- Важен порядок обработки для одинаковых ID (хэш гарантирует, что один ID → один воркер)

Локализация данных — каждому воркеру свой набор

- Каждая горутина работает со своим счётчиком в массиве
- Нет общих ресурсов → нет блокировок
- В конце значения суммируются (редкая операция)

Преимущества:

- Полное отсутствие блокировок — каждый поток пишет только в свою ячейку памяти
- Идеальная масштабируемость — производительность растёт линейно с добавлением CPU
- Локализация кэша процессора — данные воркера находятся в L1-кэше его ядра

```
var mu sync.Mutex
var counter int

func increment() {
    mu.Lock()
    counter++
    mu.Unlock()
}
```

Как работает локализация данных

Когда применять:

- Высокочастотные счётчики (статистика, метрики)
- Агрегация событий (логирование, аналитика)
- Параллельные вычисления (MapReduce-подобные задачи)

```
var mu sync.Mutex
var counter int

func increment() {
    mu.Lock()
    counter++
    mu.Unlock()
}
```

Недостатки общего счётчика с мьютексом

- Высокий contention — все горютины борются за один мьютекс
- Низкая производительность — до 80% времени уходит на ожидание блокировки
- Плохая масштабируемость — добавление CPU не ускоряет обработку

Локальный счётчик для каждого воркера

```
var counters [8]int

func worker(id int, input <-chan int) {
    for range input {
        counters[id]++
    }
}
```

Буферизация для устранения contention в логгере

```
func Log(msg string) {  
    mu.Lock()  
    defer mu.Unlock()  
    file.Write([]byte(msg))  
}
```

Недостатки синхронного логгера

- Высокий contention — все горютины ждут своей очереди для записи
- Непредсказуемые задержки — время работы `Log()` зависит от нагрузки
- Просадки производительности — при пиковых нагрузках система может «захлебнуться»

Решение: буфер + отдельный поток на запись

```
var logChan = make(chan string, 1000)

func init() {
    go func() {
        for msg := range logChan {
            file.Write([]byte(msg))
        }
    }()
}

func Log(msg string) {
    logChan <- msg
}
```

Как работает буферизация логгера

Писатели (горутины)

- Кладут сообщения в канал
- Не знают о существовании файла
- Работают на скорости 1–2 нс/операцию

Писатель (отдельная горутина)

- Последовательно обрабатывает сообщения
- Пишет в файл без конкуренции
- Может делать batch-записи для оптимизации

Буфер (канал)

- Сглаживает пики нагрузки
- Позволяет кратковременно работать быстрее скорости записи на диск

Асинхронная обработка и очереди

```
func handler(w http.ResponseWriter, r *http.Request) {  
    db.Insert(parse(r))  
    w.Write([]byte("ok"))  
}
```

Недостатки синхронной записи в БД

- Весь сервер ждёт завершения DB-операции
- Нулевая отказоустойчивость — при падении БД падает и сервер
- Плохая масштабируемость — throughput ограничен скоростью БД

Асинхронная обработка и очереди — решение

```
var dbQueue = make(chan Item, 1000)

func init() {
    for i := 0; i < 4; i++ {
        go func() {
            for item := range dbQueue {
                db.Insert(item)
            }
        }()
    }
}

func handler(w http.ResponseWriter, r *http.Request) {
    dbQueue <- parse(r)
    w.Write([]byte("ok"))
}
```

Ключевые преимущества асинхронной обработки

- Мгновенный ответ клиенту — HTTP-обработчик не ждёт завершения DB-операции
- Сглаживание пиков нагрузки — буфер на 1 000 элементов принимает всплески трафика
- Контролируемый параллелизм — 4 воркера оптимально нагружают БД, не перегружая соединения
- Отказоустойчивость — при падении БД сервер продолжает принимать запросы (пока есть место в очереди)

Архитектурные особенности

Разделение ответственности:

- HTTP-сервер — быстрая маршрутизация
- Воркеры — медленные операции

Паттерн **Producer-Consumer**

Гибкость масштабирования:

- можно динамически добавлять воркеров
- разные приоритеты через несколько очередей

Как понять, что оптимизация сработала

Инструменты:

- go test -bench — измерение времени выполнения
- pprof — mutex profile, block profile
- trace — визуализация ожидания ресурсов

Метрики:

- Latency P50/P95/P99
- Throughput
- Количество горутин в блокировке

Поведение под нагрузкой:

- Увеличение числа горутин не замедляет работу
- Throughput растёт, latency не деградирует
- CPU загружен стабильно без долгих ожиданий

Промежуточные итоги

- Устранение contention — фундамент надёжной, масштабируемой и эффективной архитектуры
- Начинаем с поиска узких мест и измерений
- Применяем структурные приёмы: шардирование, разделение каналов, локализация данных, буферизация, асинхронные очереди, RCU
- Проверяем результат под нагрузкой и в профилировщиках
- Избегаем общих ресурсов там, где это не нужно



Ваши вопросы?

Слишком много горутин



6

Почему возникает много горутин

- На каждый запрос создаётся горутина без ограничения количества
- Рекурсивный вызов или бесконечный цикл с `go func()`
- Задачи в очереди, под каждую создаётся отдельная горутина
- Чтение или запись в канал запускаются в отдельных горутинках
- Горутинки не завершаются и висят в ожидании

Признаки оверхеда горутин

- Резкий рост `runtime.NumGoroutine()`
- Высокое потребление памяти при небольшом объёме данных
- В `pprof` много заблокированных горутин
- В логах — `fatal error: all goroutines are asleep - deadlock`
- Приложение «задыхается» при росте нагрузки

Почему много горутин — это плохо

- Память — каждая горутина требует минимум 2 КБ стека, который может расти. 100 тыс. горутин могут занять гигабайты
- Планировщик Go — при сотнях тысяч горутин тратит ресурсы на переключение между ними
- GC-паузы — сборщик мусора дольше обходит стеки, особенно при ссылках на объекты в куче
- Сложность отладки — тысячи горутин можно отследить, миллионы — почти нет
- Утечки — горутины висят в ожидании данных, которые никогда не придут

Примеры бесконтрольного создания горути

```
func main() {  
    for {  
        go handleRequest()  
    }  
}
```

- Запускает бесконечно много горути без контроля
- При блокирующих операциях их число может достигать десятков тысяч

Примеры бесконтрольного создания горути

```
for _, task := range tasks {  
    go process(task)  
}
```

- При миллионе задач создаётся миллион горути
- На практике тормозит сильнее, чем последовательная обработка

Пример из практики

- **На каждый HTTP-запрос создавалась горутина, ожидающая ответ БД**
 - На тестах всё работало, но при 1000 RPS система зависала
 - В логах — десятки тысяч горутин, рост потребления памяти
- Причина — медленные ответы БД, горутин копились и не завершались
- Решение — ограничить параллелизм через worker pool и буферизированный канал
- Результат — стабильное число горутин и нормальная работа под нагрузкой

Как найти избыточные горутины

- `runtime.NumGoroutine()` — текущее количество
- `pprof` — состояние и стэки горутин. `go tool pprof`
`http://localhost:6060/debug/pprof/goroutine`
- `go tool trace` — таймлайны активности
- Логгирование запуска `go func()`
- Метрики `Prometheus` (`go_goroutines`)

Типовые причины оверхеда горутин

- Горутина на каждый запрос или сообщение
- **Goroutine leak** — канал не закрыт, завершение не предусмотрено
- Нет контроля параллелизма, отсутствует **worker pool**
- Неограниченный **select {}** с таймерами или каналами
- Запуск горутины в цикле по большому набору данных
- Необработанные ошибки — горутина падает незаметно

Промежуточный итог

- ✓ Горутины в Go — мощный инструмент, но требуют контроля
- ✓ Миллион горутин — потенциальная проблема, а не решение
- ✓ Последствия избыточного числа горутин:
 - рост потребления памяти
 - увеличение времени переключения контекстов
 - блокировки на каналах и мьютексах
 - заторможенный планировщик
 - падение производительности
 - нестабильная и трудноотлаживаемая работа
- ✓ Успешная Go-программа — это там, где горутин достаточно, но их число контролируется



Ваши вопросы?

Подходы к решению и лучшие практики

7

Не создавайте горутину без необходимости

- Горутина — инструмент, а не обязательный способ запуска кода
- Иногда достаточно синхронного вызова
- Параллелизм нужен, если реально выигрываете во времени
- Если операции быстрые или обрабатываются по очереди — горутина не нужна

Пул воркеров вместо горютины на каждую задачу

Плохой код:

```
for _, task := range tasks {  
    go process(task)  
}
```

- При 100 000 задачах создаётся 100 000 горютин
- Растёт потребление памяти
- Планировщик тормозит, система нестабильна

Пул воркеров — оптимизированный подход

```
func worker(id int, jobs <-chan int, wg
*sync.WaitGroup) {
    defer wg.Done()
    for job := range jobs {
        process(job)
    }
}

func main() {
    jobs := make(chan int, 100)
    var wg sync.WaitGroup

    numWorkers := 10
    wg.Add(numWorkers)
```

```
    for i := 0; i < numWorkers; i++ {
        go worker(i, jobs, &wg)
    }

    for _, job := range tasks {
        jobs <- job
    }

    close(jobs)
    wg.Wait()
}
```

Только 10 горутин независимо от числа задач

Задачи поступают через канал

Graceful shutdown через WaitGroup

Результат оптимизации пула воркеров

До:

- 100 000 горутин
- Память 2–4 ГБ
- Скачки нагрузки на CPU
- GC подвешивает приложение
- Ошибки: deadlock, timeout, out-of-memory

После:

- 10–50 горутин
- Память стабильна
- CPU используется предсказуемо
- Устойчивость к нагрузкам
- Код чище и управляемее

Ограничение параллелизма через семафор

```
sema := make(chan struct{}, 5) // максимум 5 одновременных операций

for _, task := range tasks {
    sema <- struct{}{}
    go func(t Task) {
        defer func() { <-sema }()
        process(t)
    }(task)
}
```

- Контроль числа параллельных задач
- Новая задача ждёт, если лимит достигнут
- Подходит для внешних API и БД с ограниченной пропускной способностью

Контекст и отмена горутин

```
func main() {  
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)  
    defer cancel()  
  
    go func(ctx context.Context) {  
        select {  
        case <-time.After(5 * time.Second):  
            fmt.Println("Done")  
        case <-ctx.Done():  
            fmt.Println("Cancelled:", ctx.Err())  
        }  
    }(ctx)  
  
    time.Sleep(3 * time.Second)  
}
```

Контекст и отмена горутин

- Позволяет задать таймаут выполнения горутины
- Даёт возможность отменить горутину, если родительская задача завершилась или отменена
- Гарантирует корректное завершение, чтобы горутина не продолжала работать зря
- Предотвращает утечки — горутина не зависает в ожидании данных, которые никогда не придут
- Подходит для запросов к БД, внешним API и долгих вычислений, где важно ограничить время работы

Буферизованные каналы как регулятор потока

```
tasks := make(chan Task, 100) // буфер ограничен  
  
go producer(tasks)  
go consumer(tasks)
```

- Ограничивают количество накопленных задач
- Producer блокируется при переполнении буфера
- Простое и эффективное регулирование нагрузки

Диагностика «до и после»

- `runtime.NumGoroutine()` — проверяйте, что число горутин стабильно
- `pprof` – `/debug/pprof/goroutine?debug=2` покажет горутин, висящие в ожидании
- Сравните профили `heap` и `cpu` до и после оптимизации
- Метрики: `go_goroutines`, `go_memstats_heap_inuse_bytes`, `go_threads`
- После внедрения пула или семафора профиль работы заметно улучшается

Канал-сигнал завершения

```
done := make(chan struct{})
```

```
go func() {  
    select {  
    case <-work:  
        // ...  
    case <-done:  
        return  
    }  
}()
```

```
// завершение  
close(done)
```

- Передаёт сигнал воркерам о завершении работы
- Можно заменить на `context.Context` для таймаутов и отмены
- Исключает утечки горутин и делает поведение кода предсказуемым

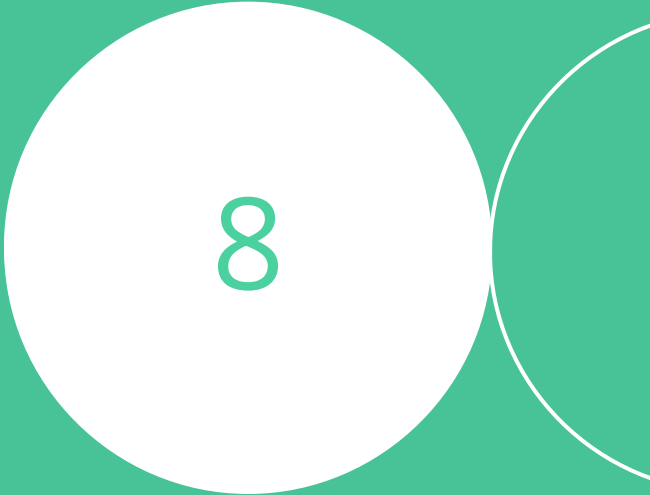
Промежуточный итог

- **Worker pool** — фиксированное число горутин и общий канал задач
- Ограничение параллелизма через семафоры (каналы с **capacity**)
- Контекст для управления временем жизни горутин
- Буферизованные каналы для регулирования потока данных
- Завершение горутин через **context**, **close()** или **WaitGroup**



Ваши вопросы?

Общие рекомендации и чек-лист



8

Чек-лист по многопоточной разработке

- 1 Не создавайте горутину без необходимости
- 2 Проверьте, действительно ли задача должна выполняться параллельно
- 3 Ограничивайте число параллельных задач
- 4 Используйте worker pool, семафоры или буферизованные каналы

Чек-лист по многопоточной разработке

- 1 Используйте `context.Context` в долгоживущих процессах
- 2 Для отмены выполнения. Для предотвращения утечек горутин
- 3 Гарантируйте завершение горутин. `sync.WaitGroup` для ожидания. `context` или `close()` для отмены и сигнализации
- 4 Соблюдайте принцип «одна ответственность — одна горутина»
- 5 Предпочитайте каналы вместо мьютексов
- 6 Профилируйте производительность с `pprof`
- 7 Документируйте и визуализируйте взаимодействие

Чек-лист — как проверить многопоточный код

- Не создаю лишние горутини — есть ли у каждой цель
- Есть контроль над числом одновременных задач — лимиты, семафоры, пул
- Все горутини завершаются — WaitGroup, context, done-каналы
- Используется context.Context — есть ли отмена
- Где можно заменить мьютекс на канал
- Нет риска гонки или deadlock
- Проведён pprof-анализ — горутини не висят, память стабильна
- Потоки данных описаны — канал откуда и куда, кто читает
- Канал может быть закрыт — нет паники при close()
- Система выдерживает нагрузку — проведены тесты на 1 000 задач

Итоги занятия

- Горутины лёгкие, но не бесплатные
- Ошибки многопоточности часто скрыты и влияют на производительность
- Разбирали и устраняли: data race, deadlock, goroutine leaks, starvation, избыточную синхронизацию, contention, оверхед от лишних горутин
- Оптимизация — архитектурная задача, а не разовая починка
- Лучшие практики и шаблоны помогают проектировать код правильно с самого начала



Ваши вопросы?

Оптимизация многопоточных программ

Матвей Ефимов

Эксперт в Go, Python, SQL

